

# Pipeline FAQ

Seven follow-up questions from reviewing docs 1–7, answered together in one place, each with a diagram.

companion to docs 1–7    2026-08-16

## 01 What does "immutable artifact" mean?

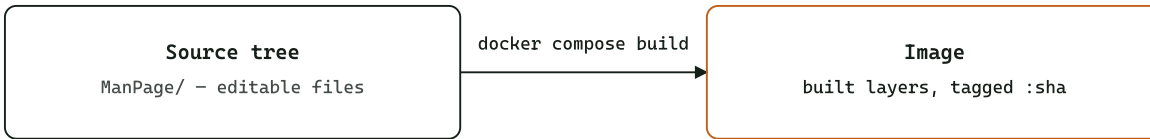
A built, versioned thing that never changes after it's created. Need something different — build a new one with a new identity, never modify the existing one in place. Same idea as a git commit: a hash never changes meaning either. The pipeline turns one into the other, tagging every image by the commit SHA it was built from.



None of these ever change once built — that's the whole property that makes the registry a reliable rollback target.

## 02 What is a source tree?

The project's actual files on disk — code, config, folders — in plain, editable form. Not a build output, not a packaged artifact. `ManPage/` itself is the source tree; `docker compose build` reads it and produces something categorically different — an image — which is no longer a source tree at all.



Only the source tree side needs Dockerfiles and dependencies present — clubuntu, in the finished design, never sees this side at all.

### 03 What does a runner actually do?

Executes a workflow's steps — GitHub never runs code itself, it hands jobs to a runner. It **polls outbound** for work rather than being pushed to, checks out source, runs each step as an ordinary shell command, and reports the result back.



Self-hosted here means this whole loop runs on clubuntu itself — so "executing" for this project means localhost calls to the registry and the deployed stack, not a network hop.

### 04 What does "resolving a tag to a SHA" mean?

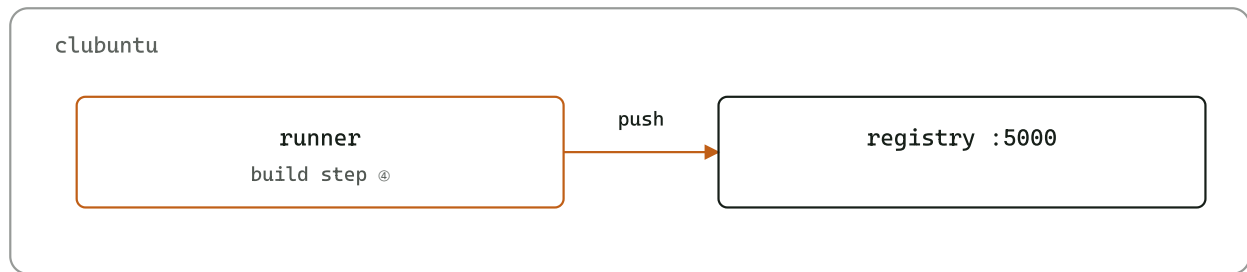
A tag ( `v1.4.0` ) is a human-readable label pointing *at* a commit — not the commit itself. The SHA is that commit's permanent identifier. Resolving looks up which SHA a tag currently points at, because the registry only understands SHA tags, never release names.



`SHA=$(git rev-list -n 1 ${GITHUB_REF_NAME})` is the whole mechanism — one lookup, no rebuild.

## 05 When, and by which component, are images pushed?

During *build* only — step ④ of the build sequence, right after `docker compose build`, triggered by every push to `main`. Never during deploy; deploy only ever pulls. The component doing the pushing is the self-hosted runner itself — no separate service.

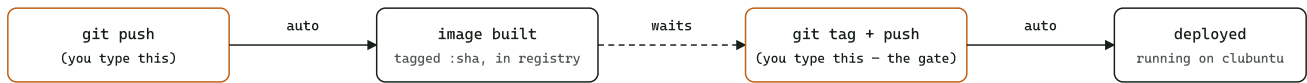


Runner → local Docker daemon → registry, entirely on one machine — the same reason no port ever needs opening for this to work.

## 06 Summary of the deployment pipeline

Immutable artifacts + CI/CD automation, with a deliberate git-tag gate before anything deploys.

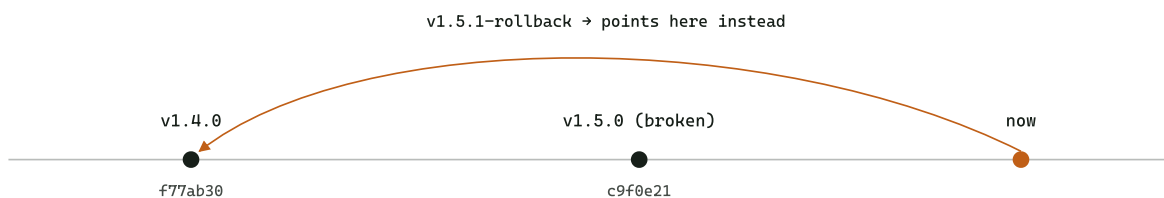
- ▶ **Build** — every push to main, automatic: runner builds five SHA-tagged images, pushes to the registry. Nothing deploys.
- ▶ **Deploy** — only on a git tag: runner resolves the tag to a SHA, pulls that SHA's images (already built), runs `up -d`.
- ▶ **Rollback** — the identical deploy sequence, given an older tag. One pull, never a rebuild.
- ▶ **Outside the pipeline entirely** — `.env`, the Postgres volume, and `coder_commands.xlsx`: persistent state on clubuntu, never baked into an image.



Two commands, both ordinary git — everything between them is automatic.

## 07 What happens during a "rollback tag"?

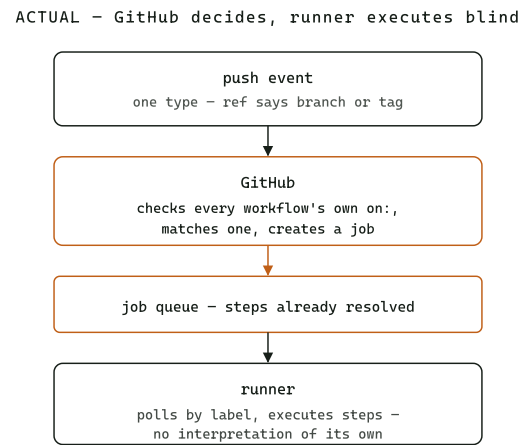
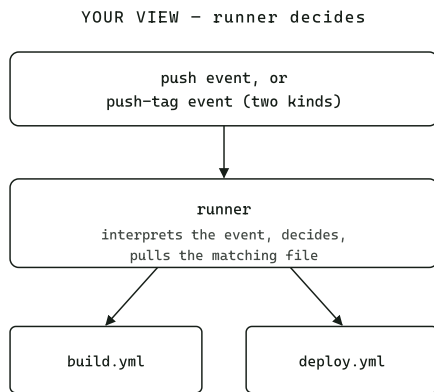
A rollback tag is a normal tag that happens to point **backward** in history — the deploy workflow has no special-case logic for it. Say `v1.4.0` (`f77ab30`) worked, `v1.5.0` (`c9f0e21`) is broken: tagging `v1.5.1-rollback` at `f77ab30` and pushing it resolves back to the old SHA, pulls its already-built images, and runs `up -d` — the same six steps as any release.



The image for `f77ab30` never went anywhere — it's been sitting in the registry since the original `v1.4.0` build. Rollback just points at it again.

## 08 Does the runner decide which workflow file to run?

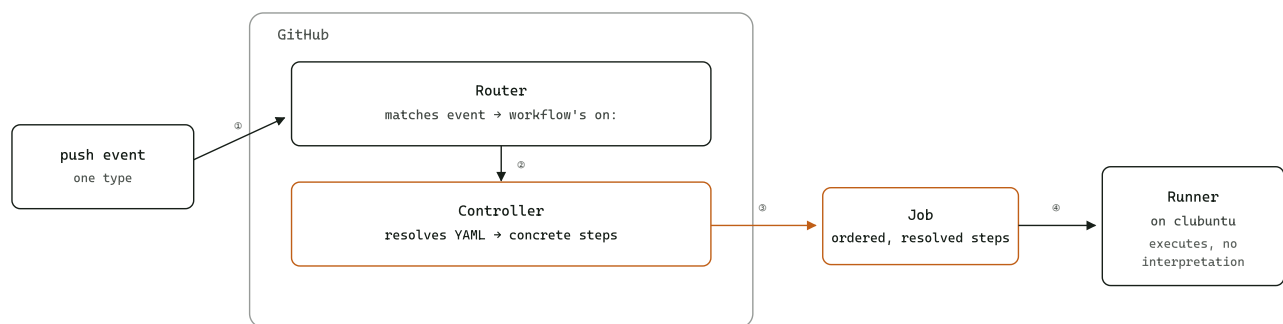
No — and this is the one place the natural mental model gets the division of labor backward.



Two corrections in one picture. First: it's one `push` event, not two kinds — branch vs. tag is just what the `ref` says. Second: matching an event to a workflow happens entirely on GitHub's side, before a runner is ever involved — the runner only label-matches a job GitHub already resolved, then executes its steps with zero awareness of why it was chosen.

## 09 Where does "router" end and "controller" begin?

Both live inside GitHub, as two stages of the same service — router matches, controller resolves. A **job** is the controller's output: the matched workflow's YAML, fully expanded into a concrete ordered list of steps with nothing left to interpret. That object — not the file — is the only thing that ever reaches a runner.



Everything left of the Job box is GitHub's own infrastructure — router and controller both. Everything right of it is yours: clubuntu polling in, collecting a plan it had no part in making, and mechanically carrying it out.

